Part 1: The Environment

The bounded knapsack problem is modeled as a Markov Decision Process (MDP), where an agent sequentially selects items to include in a knapsack with a limited capacity.

- **State:** The current state includes the remaining capacity of the knapsack and information about available items (e.g., weights and values).
- **Action:** At each step, the agent selects an item to include in the knapsack.
- **Reward:** The agent receives a reward equal to the value of the selected item, as long as the capacity constraint is not violated.
- **Transition:** The environment updates the remaining capacity after each selected item.
- **Termination:** The episode ends when no more items can be added without exceeding the capacity.

This formulation allows reinforcement learning agents to learn policies that maximize the total value of selected items.

Part 2: Training DQN and PPO

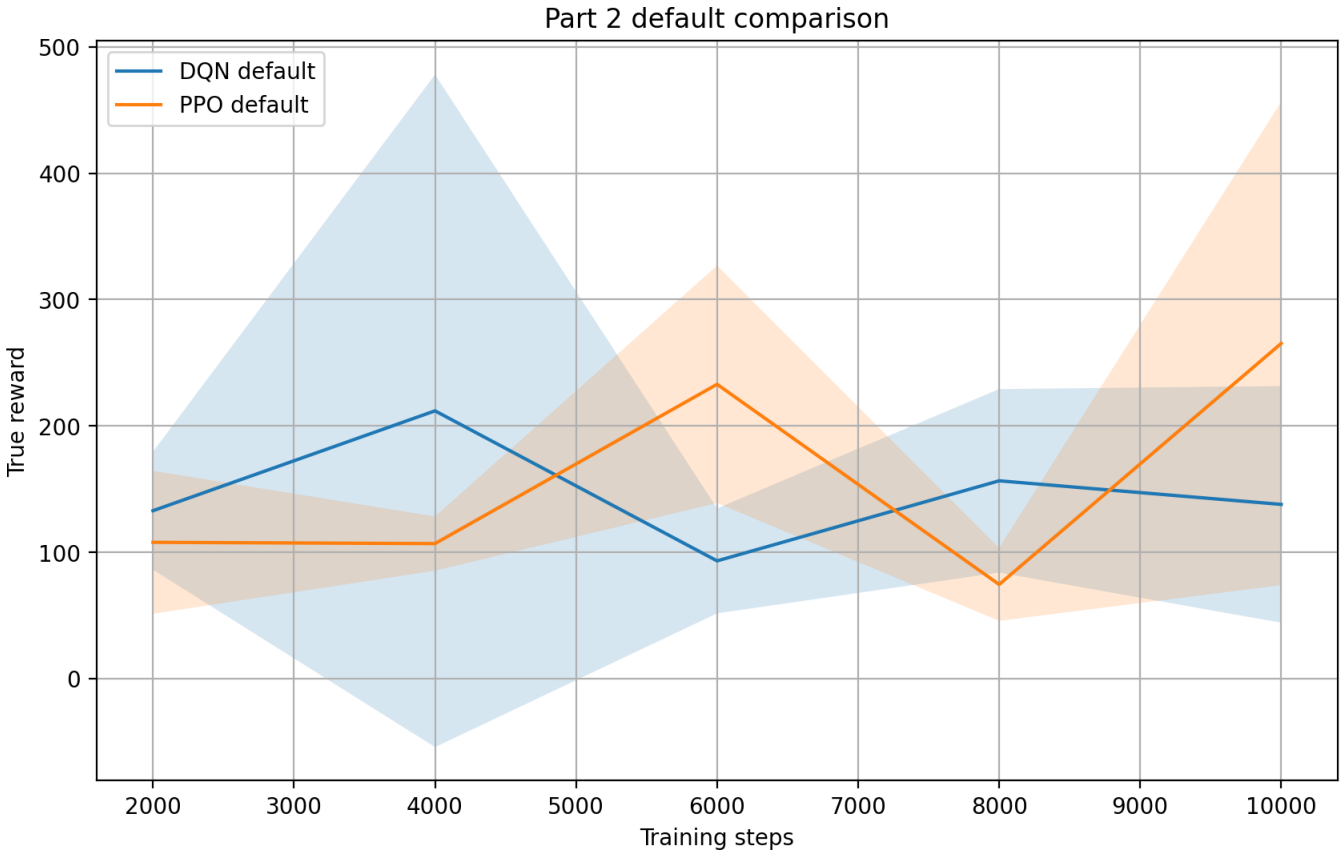
Model Explanation

DQN (Deep Q-Network) is a value-based reinforcement learning method that approximates the Q-function using a neural network. It learns by minimizing the difference between predicted Q-values and target Q-values using experience replay and a target network to stabilize training.

PPO (Proximal Policy Optimization) is a policy-based method that directly optimizes the policy using a clipped surrogate objective. This prevents large updates and ensures more stable learning compared to traditional policy gradient methods.

Results with Default Hyperparameters

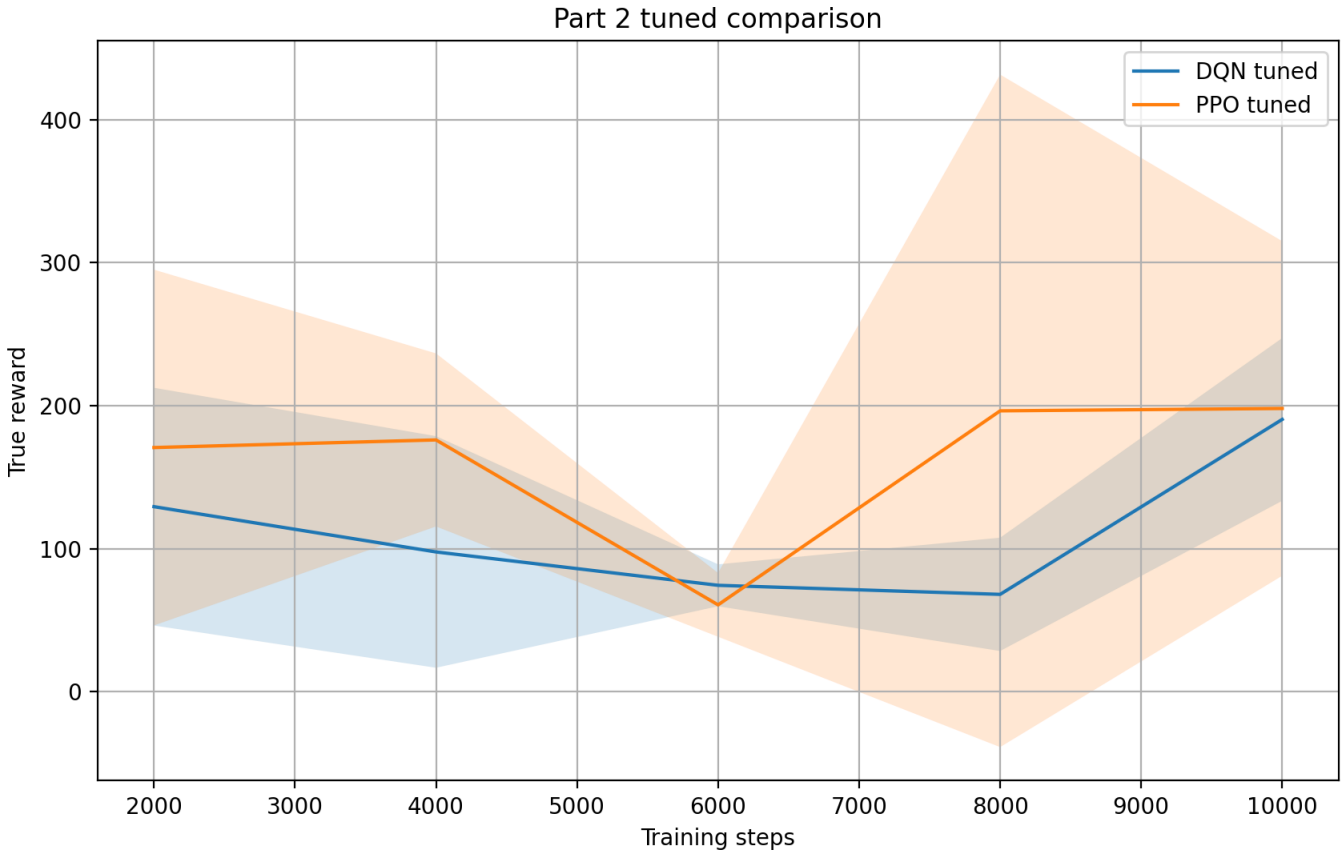
Both DQN and PPO were first trained using default hyperparameters.



The results show that DQN exhibits high variance and unstable performance across different seeds. PPO, while also showing variability, demonstrates slightly more consistent behavior.

Results with Tuned Hyperparameters

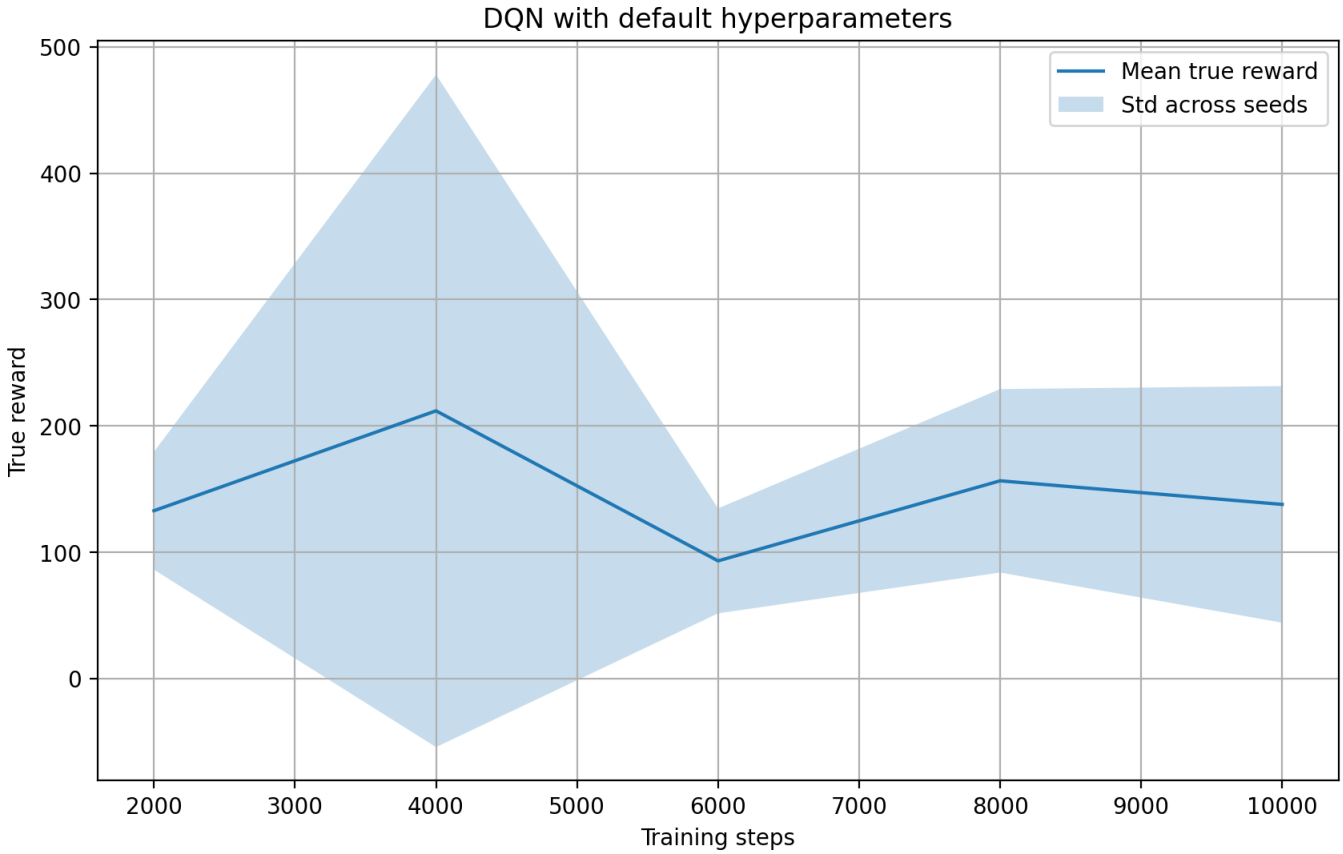
Hyperparameter tuning was performed for both models by testing multiple values for key parameters such as learning rate and training steps.

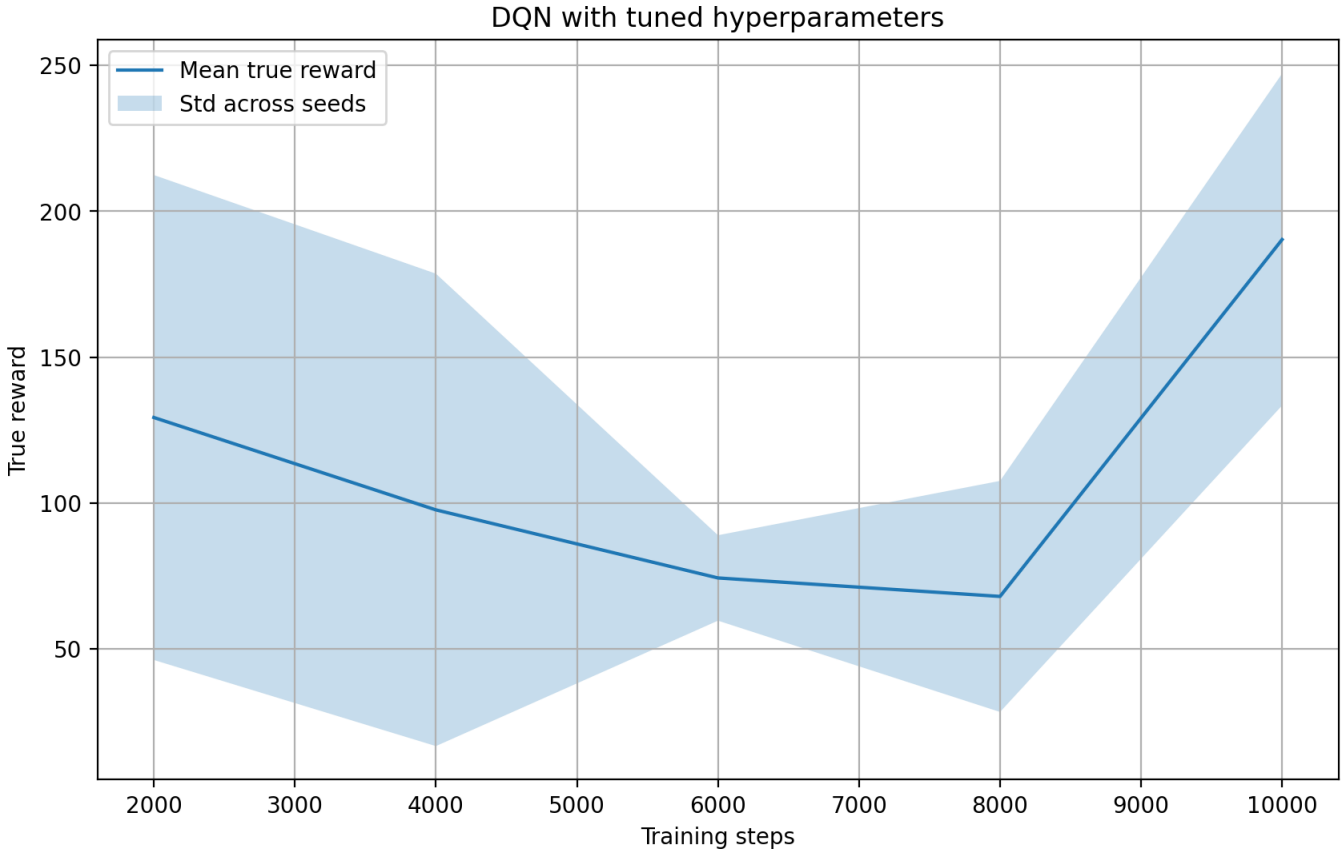


After tuning, both models show improved performance. PPO benefits more from tuning, achieving higher rewards and better stability compared to DQN.

Individual Model Behavior

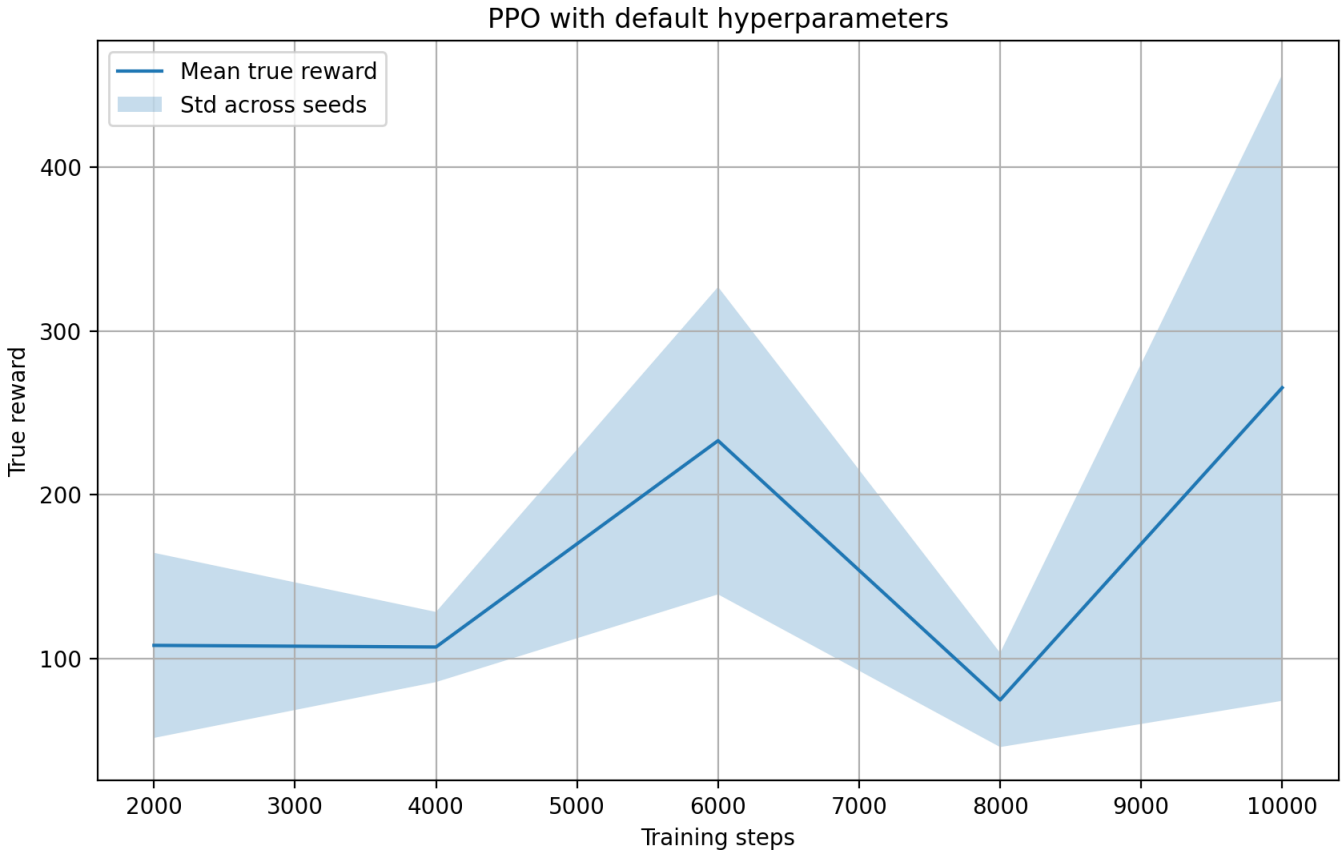
DQN (Default vs Tuned)

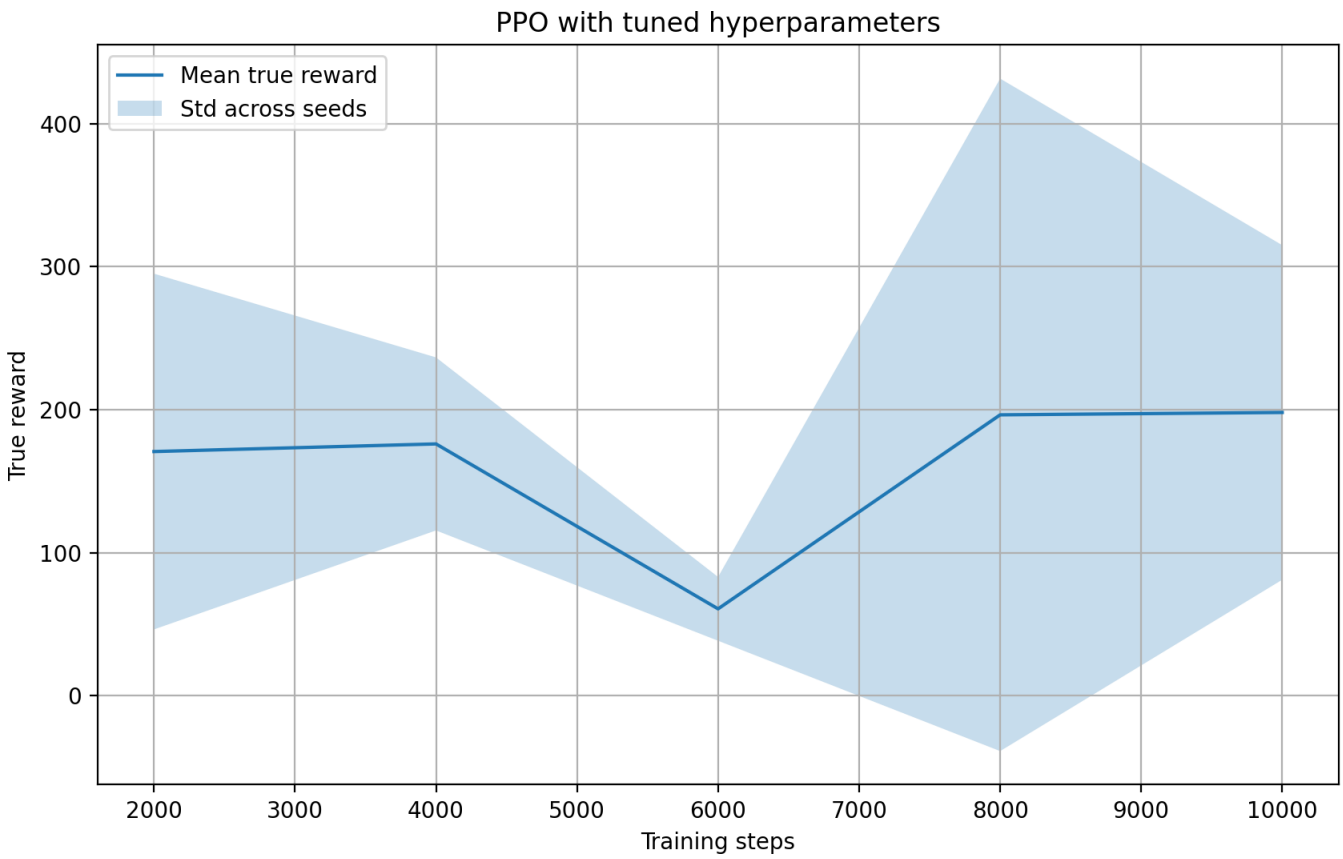




DQN shows unstable learning behavior, with large fluctuations across training steps. Even after tuning, the variance remains relatively high.

PPO (Default vs Tuned)





PPO demonstrates more stable learning compared to DQN. Tuning improves its performance, especially in later training stages.

Conclusion

Overall, PPO outperforms DQN in terms of stability and consistency. While DQN can reach competitive rewards, its high variance makes it less reliable. PPO provides more robust performance across different seeds and configurations.

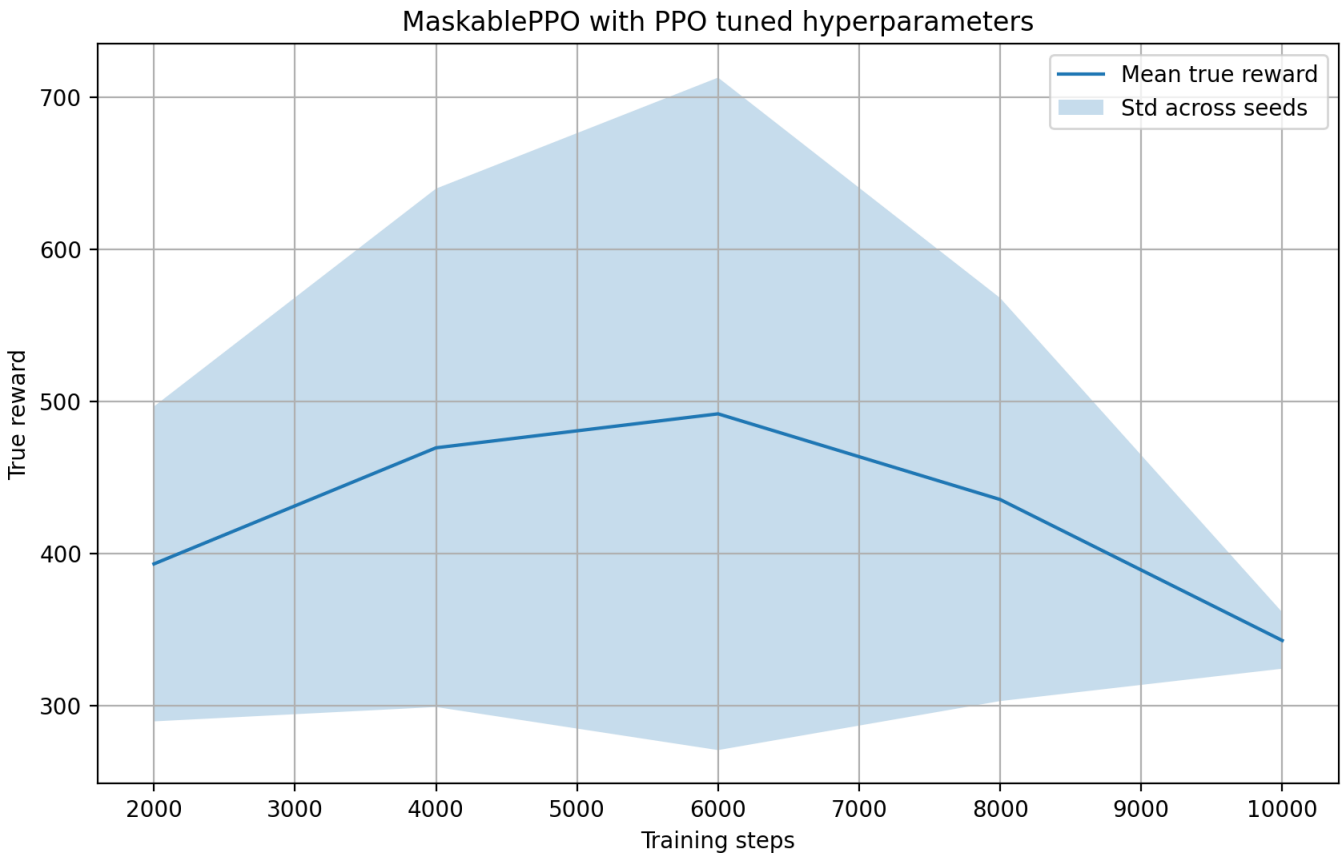
Part 3: Masked PPO

Approach

In this part, invalid action masking is applied to prevent the agent from selecting infeasible actions. The environment provides a mask that filters out invalid actions, and a MaskablePPO agent is trained using this information.

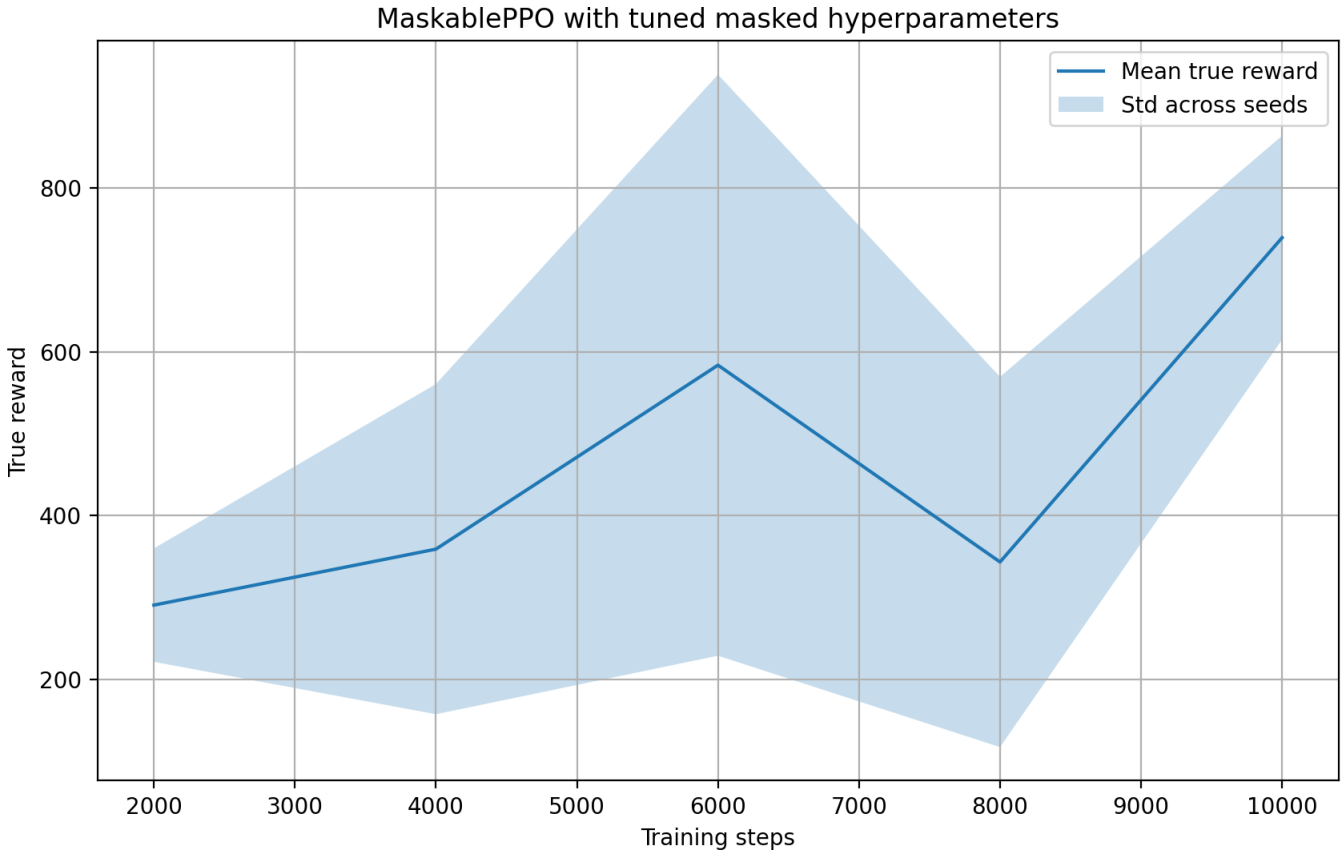
Masked PPO Results

Masked PPO using PPO tuned hyperparameters



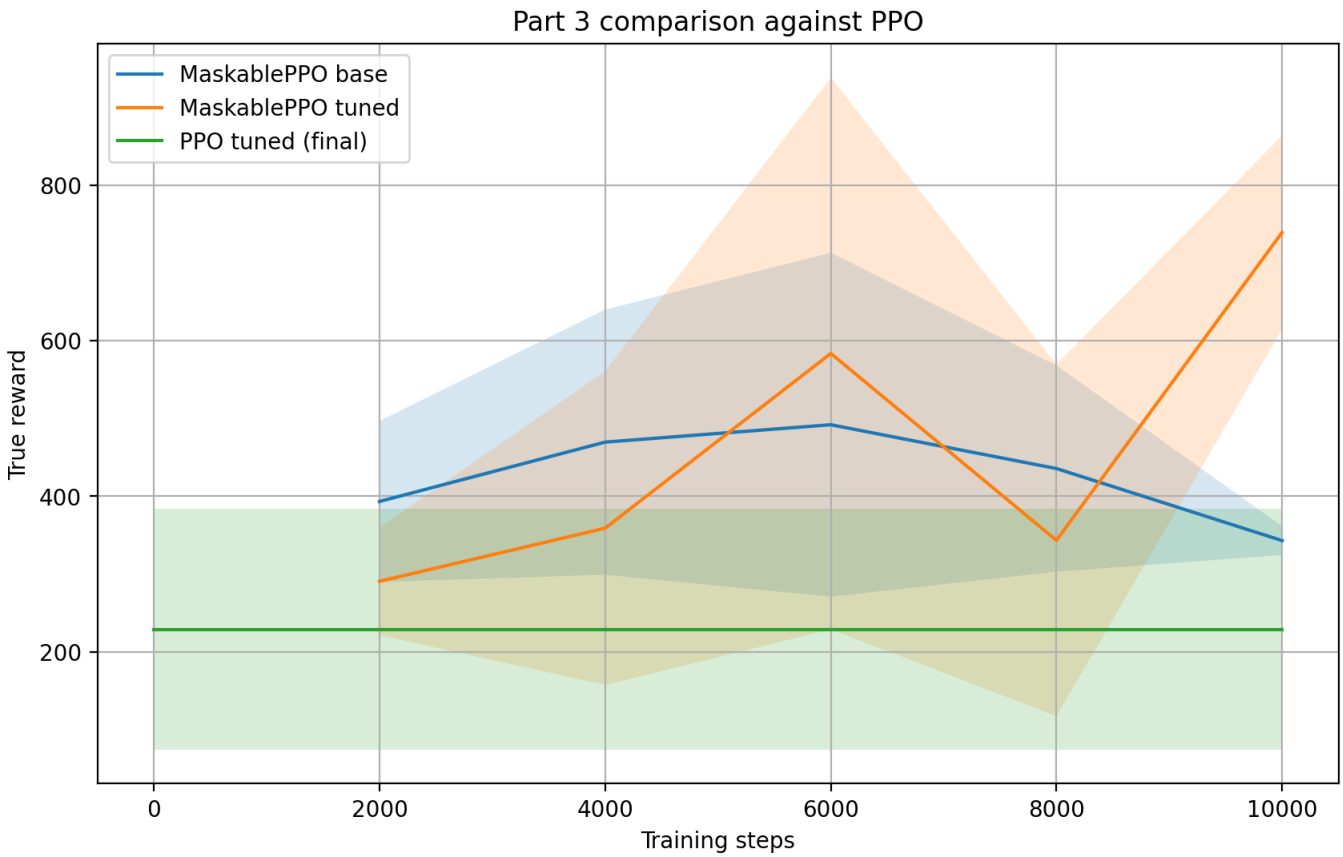
Applying masking with the PPO tuned hyperparameters already leads to a significant improvement compared to standard PPO.

Masked PPO with additional tuning



Further tuning of masked PPO hyperparameters does not significantly improve performance and may even increase variance. This suggests that the initial masked configuration was already effective.

Comparison with PPO



The comparison clearly shows that Masked PPO significantly outperforms standard PPO. The mean reward is substantially higher, and the learning process is more stable.

Conclusion

Invalid action masking greatly improves the learning efficiency of the agent by reducing the action space to only valid choices. This leads to higher rewards and more stable performance.

Interestingly, additional hyperparameter tuning does not provide major improvements, indicating that masking itself is the dominant factor in performance gains.